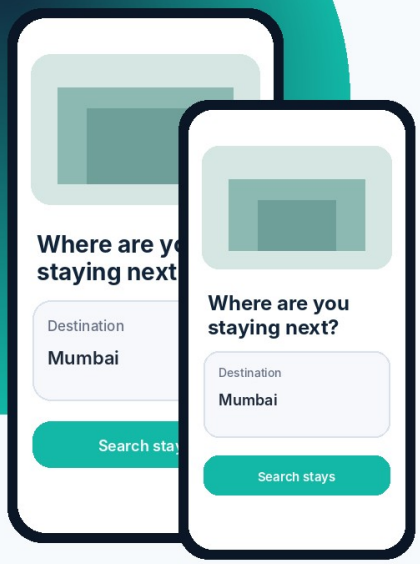
STAYEASY

Search smarter. Preview confidently. Book easier.

FINAL-YEAR PRODUCT BLUEPRINT

A complete design, engineering, AI and delivery plan for a Flutter-based hotel booking experience.

FLUTTER | MATERIAL 3 | AI CONCIERGE | 360 ROOM PREVIEW | SEASON-AWARE REVIEWS



STUDENT	PROGRAM	SEMESTER
Aniket Rai	B.Tech CSE 2024-28	V / Final-Year Project Planning

Application 34 | Problem Statement 84 | Version 1.0 | October 2026

01 / PRODUCT STRATEGY

Executive Summary

StayEasy is a Flutter-based hotel booking application designed to reduce the uncertainty travelers face while selecting a room. The project combines a complete booking workflow with two differentiators: an interactive virtual-room preview and date-aware review intelligence.

> **North-star product idea**
> Search and booking are the foundation. StayEasy wins by helping users understand what they are booking before they commit.

CORE	DIFFERENTIATOR	INTELLIGENCE
Book end-to-end	**Experience before booking**	**Relevant, not random**
Search, filters, hotels, rooms, dates, validation, reviews and confirmation.	Interactive 360-degree panorama with room-zone switching and information hotspots.	Season-aware review ranking plus grounded AI summaries and natural-language search.

Success definition

- A complete start-to-finish user journey works reliably on a real Flutter build, not only in Figma.
- The approved Figma design maps cleanly to reusable Material 3 components and responsive layouts.
- AI features solve concrete user problems and degrade gracefully when AI is unavailable.
- The Git history, tests and architecture demonstrate engineering depth suitable for a final-year evaluation.

Academic alignment

The plan directly covers the university brief: hotel dashboard, room search/detail, booking calendar, confirmation, reviews, virtual preview, Material 3 styling, Dart/Flutter logic, navigation, integration, responsive behavior and the complete Figma flow. It expands the brief with strong engineering documentation and explainable AI rather than changing the required scope.

02 / DELIVERY

Deliverables Master Plan

Every deliverable will be developed as a traceable artifact with an owner, acceptance criteria and evidence for the final demo. The goal is to avoid a last-week scramble where design, code and documentation no longer match.

Area	What we deliver	Acceptance evidence	Priority
Figma	Design system, wireframes, high-fidelity mobile screens, responsive variants, clickable prototype	Named components/variants; full happy-path prototype; loading/error/empty states	P0
Flutter UI	Material 3 screens and reusable components	Matches approved Figma; responsive; accessible interaction states	P0
Core logic	Models, filtering, availability, date validation, booking state, navigation	Unit + widget tests; deterministic behavior	P0
Reviews	Ratings, filters, seasonal prioritization,	Explainable ranking; fallback when few	P0

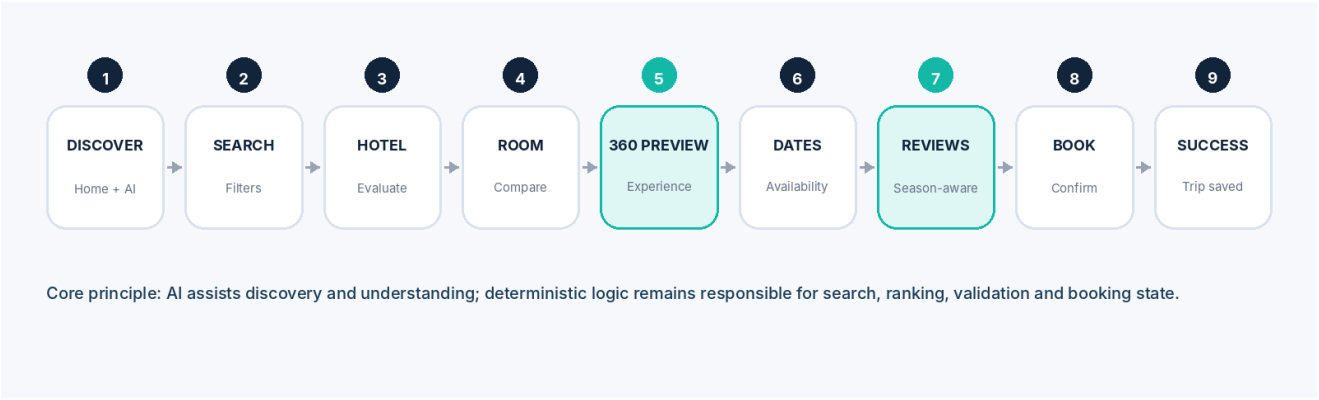
Area	What we deliver	Acceptance evidence	Priority
	room relevance	seasonal reviews exist	
Virtual preview	360 panorama viewer with room-zone controls and hotspots	Touch navigation works on target device; graceful media failure	**P0**
Backend	Auth, relational data, media storage, bookings, reviews	Seeded demo data; clear schema; safe validation	**P0**
AI	Natural-language search and review insight	Secure server proxy; structured output; grounded prompts; non-AI fallback	**P1**
Quality	Responsive QA, testing, performance, error handling	Test report + demo checklist + fixed critical issues	**P0**
Documentation	README, architecture, ERD, setup, API notes, project report	A new machine can run the app from documented steps	**P0**
Demo/Viva	Build, screenshots, story, talking points, backup recording	5-7 minute clean demo and architecture explanation	**P0**

Delivery rule

> **Definition of done**
> A feature is not complete when it merely renders. It is complete when the UI, state logic, error behavior, test coverage, responsive behavior and Git history are all in a reviewable condition.

03 / UX ARCHITECTURE

Product Experience & User Journey



The critical path deliberately follows the university-approved sequence while introducing richer decision support. The user can discover a hotel conventionally or through AI, evaluate the property, compare rooms, preview the physical room, select valid dates, inspect contextually relevant reviews and confirm the booking.

Primary personas

PERSONA A	PERSONA B	PERSONA C
Quick Booker	**Careful Researcher**	**Exploratory Traveler**

Already knows destination and dates. Needs speed, availability clarity and a short checkout.	Compares imagery, room details, policies and reviews before deciding.	Knows the vibe and constraints but not the exact filters; benefits from natural-language search.

Design principle

One product should serve all three behaviors without creating separate apps. Structured search remains first-class; AI accelerates discovery but never becomes a mandatory path.

04 / FIGMA PLAN

Figma Screen System

Figma acts as the design contract for Flutter. We will first establish foundations and components, then lock the critical mobile path, and only after that generate responsive derivatives. This avoids redesigning architecture after coding has started.

#	Screen	Purpose	Core UI
01	**Splash**	Brand entry	Logo, light motion
02	**Onboarding**	Value proposition	3 concise panels
03	**Sign In / Guest**	Authentication	Email/social/guest
04	**Home / Explore**	Search starting point	Destination, dates, guests, AI search
05	**AI Search**	Natural-language discovery	Prompt, parsed chips, edit controls
06	**Search Results**	Browse hotels	Sort, filter, hotel cards, save
07	**Filters**	Refine results	Price, rating, amenities, room type
08	**Hotel Details**	Evaluate property	Gallery, rating, amenities, location
09	**Rooms List**	Compare rooms	Room cards, availability, pricing
10	**Room Details**	Deep room evaluation	Gallery, specs, policy, 360 CTA
11	**360 Preview**	Virtual experience	Panorama, hotspots, zone selector
12	**Date Selection**	Choose valid stay	Availability calendar, totals
13	**Reviews**	Evidence before booking	Ratings, sort, seasonal context
14	**AI Review Insight**	Condense relevant feedback	Themes + supporting reviews
15	**Booking Details**	Guest information	Form + validation
16	**Booking Summary**	Final verification	Room, dates, price, policy
17	**Confirmation**	Commit booking	Dialog / processing state
18	**Booking Success**	Completion	ID, summary, next actions
19	**My Trips**	Manage bookings	Upcoming / past / cancelled
20	**Booking Detail**	Inspect reservation	Status, hotel, dates, guest
21	**Profile**	Settings	Identity and preferences

#	Screen	Purpose	Core UI
22	**Saved Hotels**	Return to favorites	Saved property cards

States that must be designed, not improvised later

- Search: loading, no results, network error, filters active, AI interpretation active.
- Room: available, limited, sold out, selected, media unavailable.
- Calendar: past/disabled, unavailable, today, selected start/end, selected range, validation error.
- Booking: valid, field error, processing, failure, success and idempotent retry.
- Reviews: default, season-prioritized, filtered, no seasonal match, AI summary loading/error.

05 / DESIGN SYSTEM

StayEasy Visual Language

The design direction should feel like premium hospitality interpreted through modern travel technology. It should be contemporary and recognizably Material 3 without looking like a default template.

COLOR **Deep navy + travel teal** Navy provides trust and structure; teal marks interactive and intelligent moments; warm gold is reserved for small highlights.	TYPE **Inter / clean sans** High legibility, strong hierarchy, restrained weights. Accessibility beats novelty.	SPACING **8-point system** Consistent 8/16/24/32 rhythm; compact 4-point adjustments only inside components.
RADIUS **Soft, not bubbly** 8 small, 12 medium, 16 large, 24-32 hero surfaces.	MEDIA **Photography-led** Rooms and hotels are decision content. Images should dominate where useful, not merely decorate.	MOTION **Meaningful only** Use motion to explain transitions, selections and loading - never to slow the booking flow.

Figma file structure

Use a single file named “StayEasy - Product Design” with pages: 00 Cover, 01 Product/UX Notes, 02 Foundations, 03 Components, 04 Wireframes, 05 Mobile UI, 06 Tablet UI, 07 Web UI, 08 Prototype, 09 States & Edge Cases, 10 Dev Handoff.

Component families

Buttons, inputs, navigation, hotel cards, room cards, review cards, rating components, search controls, filter chips, calendar states, price rows, media viewer controls, dialogs, loading skeletons and feedback banners should use variants rather than duplicated frames.

> **Flutter handoff rule**
> A visual pattern is not approved until we can name the reusable Flutter widget it will become. This keeps Figma aspirational but implementable.

06 / 3,000 CREDITS

Figma AI Credit Strategy

The 3,000-credit budget is sufficient to explore rapidly, but the credits should accelerate design decisions rather than replace design discipline. AI is best used for divergent exploration, secondary states and copy - not for generating twenty unrelated “final” screens.

Budget	Use	Why
~30%	Visual exploration	Generate 2-4 strong Home/Search/Hotel directions and select the design DNA.
~20%	Wireframe exploration	Quickly test information hierarchy before high-fidelity work.
~15%	Component ideas	Explore room cards, review patterns, filters and booking widgets.
~15%	Responsive/secondary	Generate tablet/web derivatives and lower-priority screens after mobile is locked.
~10%	UX copy	Empty states, helper text, confirmations and AI prompts.
~10%	Reserve	Protect credits for late-stage review feedback and presentation polish.

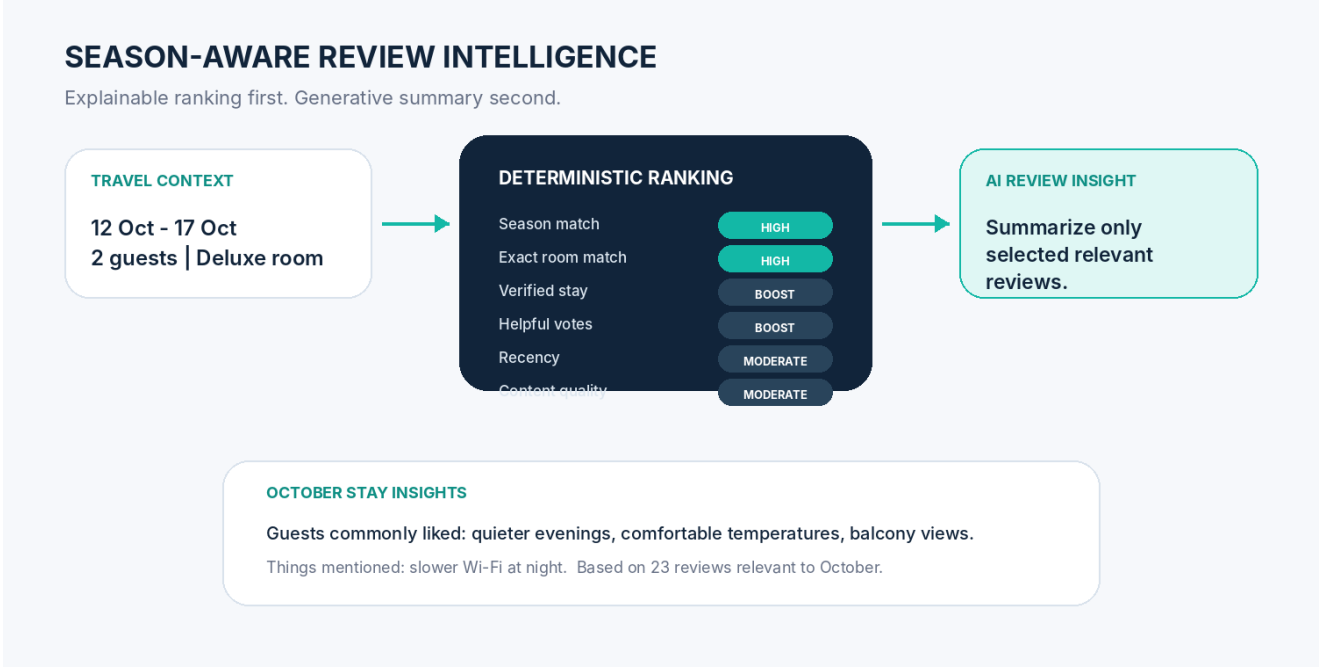
First Figma AI prompt

> **HOME / EXPLORE GENERATION PROMPT**
> Design a premium mobile hotel-booking application called StayEasy using modern Material 3 principles. Create a high-fidelity Home / Explore screen for a 390px-wide mobile device. The interface should feel modern, premium, warm, minimal and travel-focused. Avoid excessive gradients, neon styling, excessive glassmorphism and clutter. Include a greeting/profile area, “Where are you staying next?” headline, destination input, date selector, guest selector, Search button, secondary “Ask StayEasy AI” natural-language search field, recommended hotel cards, category chips, popular stays and bottom navigation. Hotel cards must show image, name, location, rating, price and useful amenities. Follow an 8pt spacing system and ensure the interface is realistic to build in Flutter.

Generate only 2-4 directions. Choose the strongest layout and then refine it manually into the system before asking Figma AI for Search Results. The second screen must inherit the selected visual language rather than starting a new style.

07 / SIGNATURE FEATURE

Season-Aware Review Intelligence



When a user searches for an October stay, reviews connected to October experiences should become more visible than unrelated seasonal feedback. The ranking remains deterministic and testable; AI is used only after relevant reviews have been selected.

Suggested scoring model

Start with an explainable weighted score rather than a black-box recommender. The exact coefficients can be tuned after seeded data is available.

Signal	Initial weight	Interpretation
Same travel month	+40	Strong seasonal relevance.
Adjacent month	+18	Useful when exact-month data is sparse.
Exact room match	+30	The reviewer experienced the same room type.
Verified stay	+12	Higher confidence than unverified content.
Helpfulness	0 to +12	Normalize useful votes so popular old reviews do not dominate.
Recency	0 to +10	Decay smoothly over time rather than hard-cutting older reviews.
Content quality	0 to +8	Reward substantive reviews using deterministic heuristics.

Cross-month stays

For a stay such as 29 October to 3 November, treat both months as relevant instead of arbitrarily assigning the booking to October. A simple first version can derive the set of months touched by the date range; a later refinement can weight each month by nights spent.

Transparency

The UI should explain why certain reviews appear first: “Reviews from similar travel periods are prioritized because hotel experiences can vary by season.” This makes the feature defensible during the viva and avoids misleading users.

08 / USEFUL, GROUNDED, SECURE

AI Strategy

StayEasy should not bolt on a generic chatbot. Each AI capability should either reduce search effort or improve understanding of existing hotel data. Core booking must continue to work without AI.

AI 01	AI 02	AI 03
Natural-language search	**Review insight**	**Room explanation**
Translate “quiet room under Rs. 4,500 with Wi-Fi and balcony” into editable structured filters.	Summarize the highest-relevance reviews selected by deterministic ranking; link back to supporting reviews.	Explain why a room matches stated requirements using only known attributes and policy data.

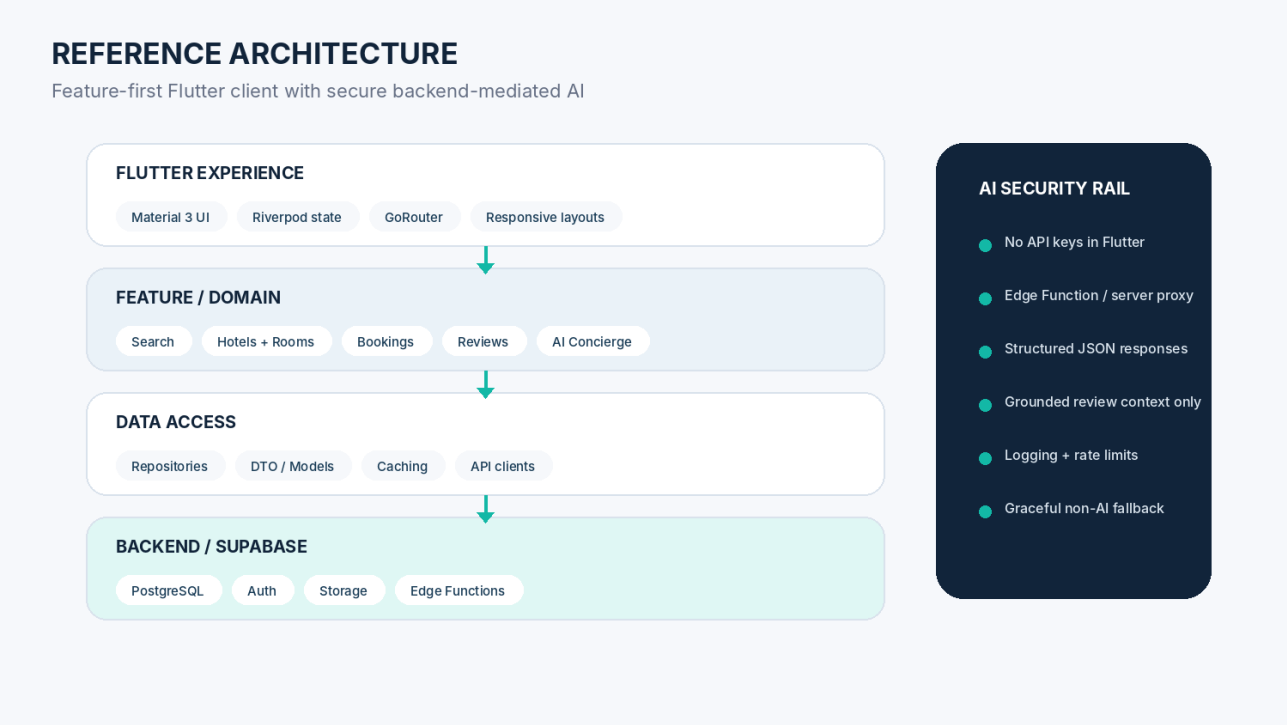
Guardrails

- Never store production AI API keys in Flutter source, .env bundled with the app or client-side remote config.
- Send requests through a secure Edge Function/backend endpoint with authentication, rate limiting and logging.
- Require structured responses (for example JSON) for search interpretation and validate them before applying filters.
- Ground review summaries only in supplied review text. Do not let the model invent hotel facilities, availability or guarantees.
- Show a conventional UI fallback whenever AI fails, times out or returns an invalid response.

> **Engineering principle**
> LLM → structured interpretation → normal application logic. The model assists; the application remains responsible for truth, validation and state.

09 / FLUTTER + BACKEND

Reference Technical Architecture



Recommended stack

CLIENT **Flutter + Dart** Android-first application with responsive tablet/web support and Material 3.	STATE **Riverpod** Predictable dependency injection and state management with good testability.	ROUTING **GoRouter** Declarative routes, deep-link-friendly structure and guarded flows.
BACKEND **Supabase** PostgreSQL, Auth, Storage and Edge Functions fit the relational booking domain.	MEDIA **Object storage** Hotel photos, room galleries and 360 panorama assets with cached URLs.	TESTING **Flutter test stack** Unit tests for logic, widget tests for behavior and one critical integration flow.

Feature-first folder direction

Use core/ for shared infrastructure and a features/ tree for home, search, hotel, room, booking, reviews, preview360, trips, profile and ai. Each feature should separate presentation, domain/data contracts and repository/service implementations as complexity requires; avoid creating empty “clean architecture” layers merely for appearance.

10 / RELATIONAL CORE

Domain & Data Model

Hotel booking is naturally relational. A PostgreSQL model keeps availability, bookings and reviews consistent while remaining easy to explain in the viva.

Entity	Key fields	Relationships / notes

User	id, name, email, phone	Has bookings, reviews and saved hotels.
Hotel	id, name, location, description, rating summary	Has rooms, amenities, media and reviews.
Room	id, hotel_id, type, capacity, base_price, status	Belongs to hotel; has media, availability and reviews.
RoomAvailability	room_id, date, status, override_price	Calendar source of truth; can support sold-out and special pricing.
Booking	id, user_id, room_id, check_in, check_out, total, status	Validated date range; must prevent conflicting confirmed bookings.
Review	id, user_id, hotel_id, room_id, stay_date, rating, body, helpful_count	Provides seasonal month from stay_date rather than posting month when possible.
Amenity	id, name, icon_key	Many-to-many with hotels/rooms.
Media	id, owner_type, owner_id, kind, url, zone	Photos and panorama assets; zone can be bedroom/bathroom/balcony.
SavedHotel	user_id, hotel_id, created_at	Unique pair to avoid duplicate favorites.

Important modeling decision

If possible, prioritize review season based on the actual stay date, not merely the date the review was posted. A traveler can post an October-stay review in November; the experience still belongs to October. If only posting dates exist in the first prototype, document the limitation explicitly.

11 / 360 EXPERIENCE

Virtual Room Preview

The requirement can be satisfied professionally without building a game engine. The first release should use equirectangular 360-degree panoramas with touch-drag interaction, fullscreen mode, room-zone switching and optional information hotspots.

ZONE
Bedroom
Primary panorama; bed, work desk, television and storage hotspots.

ZONE
Bathroom
Second panorama where available; shower/tub and amenity information.

ZONE
Balcony / View
Optional third panorama; useful for premium room categories.

Progressive enhancement

1 **Baseline**
Finger-drag panorama and zone tabs. Must work reliably on target Android device.

2 **Polish**
Fullscreen, loading skeleton, retry state, subtle orientation indicator and hotspot labels.

3 **Optional**
Gyroscope/motion navigation only if package stability and schedule allow. It is not required for project success.

> **Scope protection**
> Do not attempt a Unity-style real-time 3D room until every core booking flow is complete and tested.

12 / CONTROLLED AI DEVELOPMENT

VS Code + Codex Engineering Workflow



Codex should be treated as an implementation accelerator inside a human-controlled engineering process. Large vague prompts create inconsistent architecture and hard-to-review changes; bounded prompts keep the project maintainable.

Prompt contract for each task

- Context: current architecture, packages, relevant feature and data model.
- Objective: one clear deliverable only.
- Allowed changes: exact files/folders it may edit; forbid unrelated refactors.
- UI behavior: states, edge cases, responsive rules and accessibility expectations.
- Acceptance criteria: observable conditions that define completion.
- Tests: unit/widget/integration requirements plus commands to run.
- Report back: files changed, decisions, tests executed and remaining concerns.

Git strategy

Use short-lived feature branches or focused commits and Conventional Commit-style messages. Examples: “feat: add room availability calendar”, “feat: add seasonal review ranking”, “test: cover booking date validation”, “fix: prevent duplicate booking submission”, “refactor: extract reusable room card”. Commit after a coherent, verified unit of work - not after every keystroke and not once at the end.

13 / ENGINEERING EVIDENCE

Quality & Testing Strategy

Testing should target the logic that would embarrass us most if it failed during evaluation: dates, availability, booking state, review ranking and navigation. UI polish without deterministic behavior is not enough.

Test level	What to cover	Example evidence
Unit	Date ranges, pricing totals, filter predicates,	Fast test suite; edge cases around same-day,

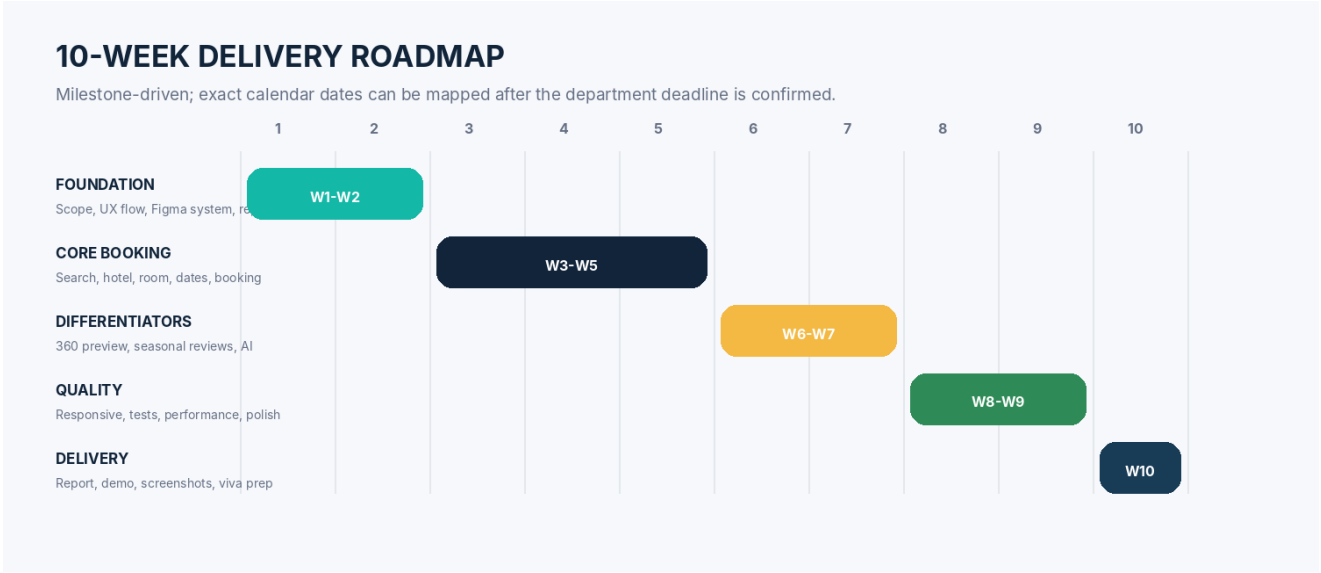
	availability checks, review scoring	past dates and cross-month review relevance.
Widget	Search/filter controls, room cards, date selection, validation messages, review sort state	Interaction tests using mocked repositories.
Integration	Search → hotel → room → 360 → dates → reviews → booking success	One deterministic seeded happy path that can run repeatedly.
Manual QA	Small/normal/large phone widths; tablet/web; offline/slow network; media failure	Screenshot checklist and issue log before final submission.

Non-functional checks

- No hardcoded secrets or production credentials in source control.
- Loading, error and empty states exist for every network-dependent screen.
- Tap targets, contrast and text scaling remain usable for accessibility.
- Images are cached/resized sensibly; long lists use lazy rendering.
- Booking submit action is protected against accidental double tap / duplicate request.
- AI timeout does not block ordinary search or review browsing.

14 / EXECUTION

Milestone Roadmap



Exit criteria by phase

A **Foundation complete**
Product scope frozen, Figma design direction selected, repository initialized, architecture decision record created, backend schema drafted.

B **Core booking complete**
Search, hotel, room, dates, availability, summary and booking success work using seeded backend data.

C **Differentiators complete**
360 viewer, seasonal review ranking and AI features are integrated with graceful fallback.

D **Quality complete**
Responsive QA, tests, error states, performance and visual consistency pass the internal checklist.

E **Submission complete**
Report, source, build, screenshots, demo flow, backup recording and viva talking points are ready.

15 / PROTECT THE PROJECT

Risk Register & Scope Control

Final-year projects usually fail from scope inflation and late integration rather than from a lack of ideas. The plan therefore treats risk management as part of engineering, not project-management decoration.

Risk	Level	Early warning	Mitigation
Figma over-design	Medium	Screens rely on effects difficult to reproduce in Flutter	Approve components only after implementation feasibility review.
Codex architecture drift	High	New packages/layers appear every task	Restrict prompt scope; inspect diff; architecture changes require explicit approval.
AI reliability	Medium	Invalid filters or fabricated summaries	Structured schemas, grounding, validation and non-AI fallback.
360 package instability	Medium	Performance/crashes on device	Prototype viewer early; keep static gallery fallback.
Booking race/conflict	High	Two bookings can overlap same room/date	Server-side availability check at confirmation, not client only.
Too much backend work	Medium	Auth/payment/API consumes project time	Seed controlled data; simulate payment unless explicitly required.
Late documentation	High	Report no longer matches implemented app	Maintain README/diagrams alongside major milestones.

16 / PRESENTATION

Final Demo & Viva Story

The final demonstration should tell one coherent story instead of clicking through screens randomly. Start with a traveler need, show how StayEasy reduces uncertainty, and end with the engineering choices that make that experience reliable.

1 **Problem**
Hotel booking requires switching between search, room details, calendars, imagery and reviews; users still struggle to judge the actual room.

2 **Discovery**

	Demonstrate normal search, then show AI translating natural language into editable filters.
3	**Evaluation** Open a property, compare rooms and launch the 360 room preview.
4	**Decision** Choose October dates and show season-aware review prioritization plus grounded AI insight.
5	**Booking** Validate dates, review pricing, confirm the reservation and show My Trips.
6	**Engineering** Explain Flutter architecture, relational data, deterministic review score, secure AI proxy, tests and Git history.

Strong viva answer: “Why not just sort reviews newest-first?”

Suggested explanation

StayEasy treats review relevance as contextual. A hotel experience can vary by season, so reviews from the user’s intended travel period receive additional weight. We combine that with exact-room relevance, verified stay, helpfulness and recency using deterministic logic. AI then summarizes the selected evidence; it does not decide booking truth or invent hotel facts.

17 / START HERE

Immediate Next Steps

The next milestone is design-system validation, not Flutter implementation. The sequence below intentionally prevents design churn and architecture churn.

1 **Create the Figma file**
Add the 11 named pages and set up the foundations page before generating application screens.

2 **Generate Home directions**
Use the approved Home/Explore prompt and produce only 2-4 strong 390px variants.

3 **Review together**
Evaluate hierarchy, typography, spacing, component reuse, Flutter feasibility and originality. Select one design DNA.

4 **Build Design System v1**
Lock color roles, type scale, spacing, radii, buttons, inputs, chips, cards and navigation variants.

5 **Design critical flow**
Complete Search → Hotel → Rooms → Room → 360 → Dates → Reviews → Summary → Success with states.

6 **Initialize engineering**
Only after the critical flow is stable: create Flutter repo, branch policy, lints, routing shell and the first Codex task prompt.

Checkpoint
Do not spend the 3,000 Figma AI credits generating the entire app before the visual system is selected. The first objective is one coherent design language.

REFERENCE

Appendix: University Brief

Original project brief supplied for Application Name 34: StayEasy / Problem Statement 84. This blueprint is intentionally aligned to its stated objectives, outcomes and deliverables.

ITM SKILLS UNIVERSITY

SCHOOL OF FUTURE TECH
ITM Skills University

B.Tech Computer Science Engineering and AI
Cross Platform Application

Application Name 34 : StayEasy
Problem Statement :84

Hotel booking can become difficult when travelers need to search rooms, select dates, evaluate reviews and understand room options before making a reservation. StayEasy will provide a centralized Flutter interface for room search, booking calendars, customer reviews and virtual room previews.

Objectives :

- UI/Widgets: Build hotel dashboard, room search, room detail, booking calendar, booking confirmation, reviews and virtual room preview screens. Use SearchBar, Filters, Cards, Lists, Calendar/Date Pickers, Rating components, Image/Media viewers, Buttons, Dialogs and navigation.
- Styling/Theming: Apply Material 3 with clear room availability and booking states, accessible typography, high-quality room presentation, rating indicators, consistent cards and responsive layouts.
- Dart/Flutter Logic: Create models for hotels, rooms, bookings and reviews; implement room filtering, date validation, booking availability, review display, virtual preview handling, booking state and navigation.
- Figma: Design the complete flow from room search → view room details → preview room → select dates → review information → confirm booking.

Outcomes :

- Develop a functional Flutter prototype demonstrating the core workflow of a hotel booking platform.
- Apply Dart and Flutter concepts to create structured, interactive, maintainable and responsive application screens.
- Translate the approved Figma user flow into a consistent Material 3 interface with appropriate widgets, booking validation and state handling.
- Demonstrate room search, booking, reviews and virtual room preview through a complete start-to-finish user journey.

Deliverables :

- Figma Design: Hotel dashboard, room search/detail, virtual room preview, booking calendar, confirmation, reviews and navigation flow.
- UI/Widgets: Room cards, search/filter controls, calendar, booking forms, rating components, virtual preview/media interface, dialogs and navigation.
- Styling/Theming: Material 3 theme, availability/booking indicators, typography, room imagery, rating presentation and responsive behavior.
- Dart/Flutter Logic: Hotel/room/booking/review models, filtering, date validation, availability handling, review display, virtual preview state and navigation.
- Integration: Integrate hotel/room availability, booking backend, review data and virtual room media/preview services.
- Responsive Prototype: Complete working journey from searching rooms through previewing, selecting dates, reviewing information and completing a booking.